Programiranje ograničenja – specijalni kurs –

Milan Banković

*Matematički fakultet,
Univerzitet u Beogradu

Prolećni semestar 2021/22.

Pregled

1 O programiranju ograničenja

2 Problem zadovoljenja ograničenja

3 Modelovanje kroz primere

O programiranju ograničenja

Šta je programiranje ograničenja?

Programiranje ograničenja je deklarativna programska paradigma koja omogućava rešavanje široke klase problema iz različitih domena.

Kakve probleme razmatramo?

Iako je gornja definicija prilično opšta, najčešće se radi o različitim problemima kombinatorne pretrage i kombinatorne optimizacije. U širem smislu, u domen programiranja ograničenja spadaju i sledeći problemi:

- rešavanje jednačina i sistema jednačina
- linearno i celobrojno programiranje
- SAT problem (iskazna zadovoljivost) i SMT problem (zadovoljivost u teoriji prvog reda)
- ...

Modelovanje problema

Na koji način se predstavlja problem koji se rešava?

Problem koji rešavamo predstavljamo skupom promenljivih koje uzimaju vrednosti iz zadatih domena:

- rešenje problema se predstavlja vrednostima koje se dodeljuju promenljivama
- uslovi koje rešenje mora da zadovoljava zadaju se ograničenjima (relacijama nad domenima promenljivih)

Rešavanje problema

Kako se rešavaju ovakvi problemi?

U zavisnosti od vrste domena i ograničenja koja se u problemu javljaju, za rešavanje se koriste različite tehnike zasnovane na veštačkoj inteligenciji (rezonovanju, pretrazi), ali i druge matematičke tehnike.
Razlikujemo:

- metode specifične za domen – pogodne za rešavanje specifičnih problema
- opšte metode – pogodne za široku klasu problema

Pregled

1 O programiranju ograničenja

2 Problem zadovoljenja ograničenja

3 Modelovanje kroz primere

Problem zadovoljenja ograničenja

Definicija 1

Problem zadovoljenja ograničenja (engl. Constraint Satisfaction Problem (CSP)) je uređena trojka $\mathcal{P} = (\mathcal{X}, \mathcal{D}, \mathcal{C})$, pri čemu je:

- $\mathcal{X} = (x_1, \ldots, x_n)$ *niz promenljivih*
- $\mathcal{D} = (D_1, \ldots, D_n)$ *niz odgovarajućih domena promenljivih, pri čemu je D_i domen promenljive x_i (što ćemo pisati kao $D(x_i) = D_i$)*
- $\mathcal{C} = (C_1, \ldots, C_k)$ *niz ograničenja, pri čemu je svako ograničenje C_i podskup od $D_{i_1} \times \ldots \times D_{i_r}$, za neki rastući niz indeksa $i_1 < \ldots < i_r$. Kažemo da je C_i ograničenje nad promenljivama $x_{i_1}, \ldots, x_{i_r}$ (u oznaci $\mathcal{X}(C_i) = (x_{i_1}, \ldots, x_{i_r})$). Broj r nazivamo arnost ograničenja C*

Rešenje CSP problema $\mathcal{P}$ je uređena n-torka $(d_1, \ldots, d_n) \in D_1 \times \ldots \times D_n$, takva da za svako ograničenje $C_i \in \mathcal{C}$ za koje je $\mathcal{X}(C_i) = (x_{i_1}, \ldots, x_{i_r})$, r-torka $(d_{i_1}, \ldots, d_{i_r})$ pripada C_i.

Napomena

- Rešenje $(d_1, \ldots, d_n)$ ćemo često zapisivati i kao dodelu $\{x_1 = d_1, \ldots, x_n = d_n\}$
- „Rešiti CSP problem" obično podrazumeva pronalaženje jednog rešenja
  - ređe, može se zahtevati pronalaženje svih rešenja

Predstavljanje ograničenja

Kako predstavljamo ograničenja?

- ograničenja možemo predstavljati skupom r-torki koje ga zadovoljavaju (tzv. tabelarna ograničenja)
- češće, ograničenja predstavljamo simbolički, kao relacije nad promenljivama (npr. $x < y$)
- ograničenja mogu biti unarna (npr. $x > 0$, $x \neq 3$, ...), binarna (npr. $x < y$, $x = y$, $x \mid y$, ...), kao i arnosti veće od dva (npr. $x + y = z$)
- vrlo česta su različita aritmetička ograničenja (npr. $x^2 + y^2 \leq 9$)
- pored jednostavnih relacija, često koristimo i ograničenja sa složenom semantikom koja nazivamo globalna ograničenja (više o njima kasnije)

Domeni promenljivih

Kakvi mogu biti domeni promenljivih?

Tehnike rešavanja CSP problema umnogome zavise od prirode domena promenljivih koje se razmatraju:

- konačni domeni: problemi kombinatorne pretrage
  - dobro razvijene opšte metode rešavanja, zasnovane na rezonovanju i pretrazi
- beskonačni domeni – diskretni i kontinualni
  - uglavnom se koriste specifične tehnike, u zavisnosti od tipa ograničenja koja se javljaju

Napomena

Mi ćemo se u nastavku baviti isključivo problemima nad konačnim domenima i opštim tehnikama za njihovo rešavanje.

Primer – Pitagorine trojke

Primer 1

Razmotrimo CSP problem $\mathcal{P}$ nad promenljivama $\mathcal{X} = (x, y, z)$, gde je $D(x) = D(y) = D(z) = \mathbb{N}$ (skup prirodnih brojeva), a skup ograničenja je zadat sa:

- $z^2 = x^2 + y^2$
- $x < y$

Ovaj problem je nad beskonačnim domenima. Rešenja ovog problema su sve Pitagorine trojke prirodnih brojeva i ima ih beskonačno mnogo. Uslov $x < y$ sprečava da pored rešenja $\{x = 3, y = 4, z = 5\}$ imamo i rešenje $\{x = 4, y = 3, z = 5\}$, što su suštinski iste Pitagorine trojke. Ovakvi uslovi se koriste za razbijanje simetrija (engl. symmetry breaking).

Primer – kriptoaritmetika

Primer 2

Razmotrimo sledeći problem: slova treba zameniti dekadnim ciframa (ista slova istim ciframa, a različita različitim) tako da sabiranje bude ispravno:

$$\begin{array}{rr} & SEND \\ + & MORE \\ \hline & MONEY \end{array}$$

Ovakvi problemi poznati su i kao kriptoaritmetičke slagalice (engl. cryptoarithmetic puzzles). Problem se može jednostavno predstaviti kao CSP problem na sledeći način:

- *Skup promenljivih* $\mathcal{X} = (S, E, N, D, M, O, R, Y)$
- *Domeni svih promenljivih je skup* $\{0, 1, 2, \ldots, 9\}$*, izuzev promenljive* M*, čiji je domen* $\{1, 2, \ldots, 9\}$ *(primetimo da su svi domeni konačni skupovi)*
- *Skup ograničenja se sastoji iz sledećih ograničenja:*
  - $S \neq E$, $S \neq N$, $\ldots$, $S \neq Y$, $E \neq N$, $\ldots$, $R \neq Y$
  - $$\begin{aligned} & 1000 \cdot S + 100 \cdot E + 10 \cdot N + D \\ & 1000 \cdot M + 100 \cdot O + 10 \cdot R + E \\ = {} & 10000 \cdot M + 1000 \cdot O + 100 \cdot N + 10 \cdot E + Y \end{aligned}$$

Primer – linearni sistem

Primer 3

Razmotrimo CSP problem nad promenljivama $\mathcal{X} = (x, y)$, *gde je* $D(x) = D(y) = \mathbb{R}$ *(skup realnih brojeva), a ograničenja su zadata na sledeći način:*

- $2x + 3y = 5$
- $x - y = 10$

Ovaj problem je nad kontinualnim domenima, a prostor rešenja je neprebrojiv. Ipak, ovaj problem ima tačno jedno rešenje: $\{x = 7, y = -3\}$.

Napomena

Ovaj primer nam pokazuje da se i poznati problem rešavanja sistema linearnih jednačina nad poljem realnih brojeva može razumeti kao specijalni slučaj CSP problema.

Problem zadovoljenja ograničenja

Definicija 2

Za CSP problem $\mathcal{P}$ kažemo da je konzistentan ako ima bar jedno rešenje. U suprotnom je nekonzistentan.

Kakav može biti skup rešenja?

- Konzistentan CSP može imati jedno ili više rešenja
- Konzistentan CSP nad konačnim domenima uvek ima konačno mnogo rešenja

Primer – nekonzistentan problem

Primer 4

Neka je problem $\mathcal{P}$ nad skupom promenljivih x, y, z, takvih da je $D(x) = D(y) = D(z) = \{1, 2\}$. Ako imamo skup ograničenja $x \neq y$, $x \neq z$, $y \neq z$, tada je jasno da je problem $\mathcal{P}$ nekonzistentan.

Pregled

1 O programiranju ograničenja

2 Problem zadovoljenja ograničenja

3 Modelovanje kroz primere

Modelovanje problema

Šta je CSP modelovanje?

Pod CSP modelovanjem podrazumevamo formulaciju datog problema iz prakse u obliku CSP problema. Dobijeni CSP problem nazivamo modelom datog problema iz prakse.

Izazovi prilikom modelovanja

- Prilikom modelovanja je ključno prepoznati promenljive i njihove domene, kao i izraziti odgovarajuća ograničenja
- Modelovanje nije trivijalan posao:
  - mnogi problemi se mogu modelovati na više načina
  - neki modeli omogućavaju jednostavniju formulaciju ograničenja, a neki efikasnije rešavanje (ne obavezno isti modeli!)

Ograničenja poređenja

Definicija 3

Ograničenja oblika $x \bowtie c$ *ili* $x \bowtie y$*, gde su* x *i* y *promenljive,* c *konstanta, a* $\bowtie \in \{=, \neq, <, >, \leq, \geq\}$ *nazivamo ograničenja poređenja.*

Napomena

Ograničenja poređenja su najjednostavnija ograničenja koja se pojavljuju u CSP problemima i veoma su često korišćena.

Primer – takmičenje

Primer 5

Petao, Gavran i Kukavica se takmiče u pevanju. U žiriju se nalazi određeni broj sudija. Svaki sudija je glasao za jednog od takmičara. Detlić je bio zadužen za brojanje glasova. Ukupno je izbrojao 59 glasova. Po njegovom brojanju, 15 ih je glasalo za Petla ili Gavrana, 18 za Gavrana ili Kukavicu, a 20 za Kukavicu ili Petla. Detliću brojanje nije jača strana, tako da ni jedan od ova četiri broja ne mora biti tačan, ali se zna da ni u jednom slučaju nije pogrešio za više od 13. Koji su rezultati glasanja?

Primer – takmičenje (2)

Primer (nastavak)

(nastavak) Uvedimo promenljive P, G i K koje označavaju redom broj glasova za Petla, Gavrana i Kukavicu. Uzmimo da je domen ovih promenljivih $\{1, \ldots, 72\}$ (s obzirom da zbir sve tri promenljive sigurno nije veći od $59 + 13 = 72$). Sada imamo sledeća ograničenja:

- $46 \leq P + G + K \leq 72$
- $2 \leq P + G \leq 28$
- $5 \leq G + K \leq 31$
- $7 \leq K + P \leq 33$

Linearno ograničenje

Definicija 4

Ograničenje oblika:

$$a_1x_1 + \ldots a_n + x_n \bowtie c$$

gde su $a_1, \ldots, a_n$ *i* c *konstante,* $x_1, \ldots, x_n$ *promenljive, a* $\bowtie \in \{=, \neq, <, >, \leq, \geq\}$ *nazivamo linearno ograničenje (ograničenje sume, ograničenje skalarnog proizvoda).*

Napomena

Ovo ograničenje je jedno od najčešćih ograničenja koja se javljaju u modelima.

Napomena

Često se javlja i ograničenje oblika:

$$y = a_1x_1 + \ldots + a_nx_n$$

koje je specijalni slučaj linearnog ograničenja (jer je ekvivalentno sa $y - a_1x_1 - \ldots - a_nx_n = 0$).

Primer – magični kvadrat

Primer 6

Dat je magični kvadrat:

	12		
	8	15	
7		2	
4			11

Potrebno je prazna polja popuniti celim brojevima iz intervala $[1, 16]$*, tako da svi brojevi u tablici budu različiti, a zbir po svim vrstama, kolonama i velikim dijagonalama bude 34.*

Primer – magični kvadrat (2)

Primer (nastavak)

(nastavak) Magični kvadrat se jednostavno predstavlja kao CSP problem:

- *Uvodimo promenljive* x_{ij} *za polje u* i*-toj vrsti i* j*-toj koloni* $(1 \leq i, j \leq 4)$
- *Sve promenljive imaju domen* $\{1, 2, \ldots, 16\}$
- *Skup ograničenja:*
  - $x_{i1} + \ldots + x_{i4} = 34$, *za svako* $i \in \{1, 2, 3, 4\}$
  - $x_{1j} + \ldots + x_{4j} = 34$, *za svako* $j \in \{1, 2, 3, 4\}$
  - $x_{11} + x_{22} + x_{33} + x_{44} = 34$
  - $x_{41} + x_{32} + x_{23} + x_{14} = 34$
  - $x_{12} = 12$, $x_{22} = 8$, $x_{23} = 15$, $x_{31} = 7$, $x_{33} = 2$, $x_{41} = 4$, $x_{44} = 11$
  - $\texttt{alldiff}(x_{11}, \ldots, x_{14}, x_{21}, \ldots, x_{24}, x_{31}, \ldots, x_{34}, x_{41}, \ldots, x_{44})$

Definicija 5

Ograničenje $\texttt{alldiff}(x_1, \ldots, x_k)$ *definišemo na sledeći način:*

$$\texttt{alldiff}(x_1, \ldots, x_k) = \{(d_1, \ldots, d_k) \mid d_i \in D(x_i), i \neq j \Rightarrow d_i \neq d_j\}$$

Intuitivno, ovo ograničenje zahteva da sve navedene promenljive uzimaju međusobno različite vrednosti.

Primer – latinski kvadrat

Primer 7

Razmotrimo primer latinskog kvadrata:

1			
			3
	4		
		2	

Prazna polja u kvadratu treba popuniti brojevima od 1 *do* 4*, tako da u svakoj vrsti i svakoj koloni svi brojevi budu međusobno različiti.*

Primer – latinski kvadrat (2)

Primer (nastavak)

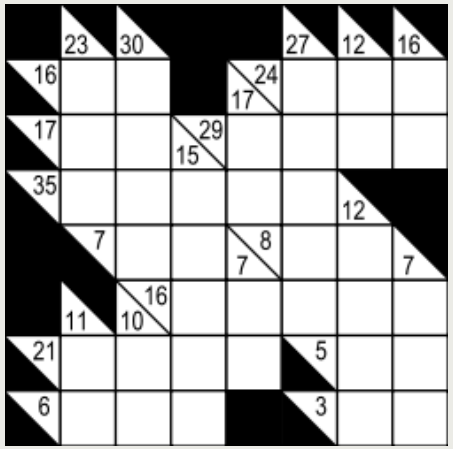
(nastavak) Latinski kvadrat se svodi na sledeći CSP problem:

- *skup promenljivih:* x_{ij} *za polje u i-toj vrsti i j-toj koloni*
- *domeni svih promenljivih su* $\{1, 2, 3, 4\}$
  - $\texttt{alldiff}(x_{i1}, \ldots, x_{i4})$, *za* $i \in \{1, 2, 3, 4\}$
  - $\texttt{alldiff}(x_{1j}, \ldots, x_{4j})$, *za* $j \in \{1, 2, 3, 4\}$

Primer – kakuro

Primer 8

Kakuro predstavlja poznatu japansku slagalicu, nalik ukrštenici, samo sa brojevima:



Bela polja treba popuniti brojevima od 1 *do* 9 *tako da se svaka „brojevna reč" po vrstama i kolonama sastoji iz različitih brojeva, a zbirovi brojeva u svakoj reči su jednaki datim brojevima.*

Primer – kakuro (2)

Primer (nastavak)

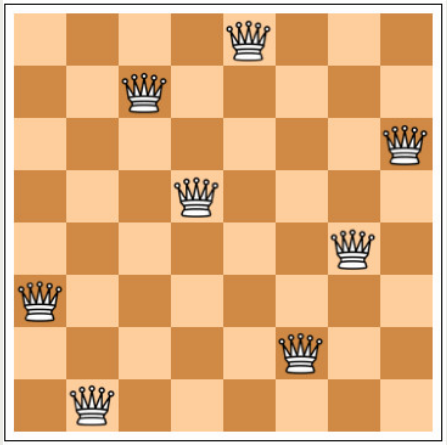
(nastavak) Kakuro slagalica iz prethodnog primera se može predstaviti kao CSP problem na sledeći način:

- *svakom belom polju se pridruži jedna promenljiva sa domenom* $\{1, 2, \ldots, 9\}$
  - *neka su promenljive pridružene belim poljima redom po vrstama, sa leva na desno* $x_1, x_2, \ldots, x_{36}$
- *za svaku „reč", promenljive koje odgovaraju poljima te reči su ograničene* `alldiff` *ograničenjem*
  - *na primer, za prvu vrstu gornje tablice imamo dva ograničenja:* $\texttt{alldiff}(x_1, x_2)$, $\texttt{alldiff}(x_3, x_4, x_5)$
- *za svaku „reč", zbir vrednosti promenljivih koje odgovaraju poljima te reči mora biti jednak datom zbiru*
  - *na primer, za prvu vrstu gornje tablice, imamo dva ograničenja:* $x_1 + x_2 = 16$, $x_3 + x_4 + x_5 = 24$

Primer – 8 kraljica

Primer 9

Razmotrimo problem 8 kraljica: potrebno je rasporediti 8 kraljica na šahovsku tablu tako da se međusobno ne napadaju. Jedno moguće rešenje ovog problema dato je na slici:



Primer – 8 kraljica (2)

Primer (nastavak)

(nastavak) Problem 8 kraljica se može predstaviti na sledeći način:

- *neka je* x_i *(za* $1 \leq 1 \leq 8$*) promenljiva koja označava redni broj vrste u kome se nalazi i-ta kraljica* ($D(x_i) = \{1, \ldots, 8\}$)
- *neka je* y_j *(za* $1 \leq j \leq 8$*) promenljiva koja označava redni broj kolone u kojoj se nalazi j-ta kraljica* ($D(y_j) = \{1, \ldots, 8\}$)
- *mora da važi:* $\texttt{alldiff}(x_1, \ldots, x_8)$ *i* $\texttt{alldiff}(y_1, \ldots, y_8)$ *(nikoje dve kraljice ne smeju biti u istoj vrsti ni u istoj koloni)*
- *za* $i \neq j$ *mora da važi:* $|x_i - x_j| \neq |y_i - y_j|$ *(nikoje dve kraljice ne smeju biti na istoj dijagonali)*

Primer – 8 kraljica (3)

Primer (nastavak)

(nastavak) Prethodni model problema 8 kraljica se može pojednostaviti tehnikom razbijanja simetrija: s obzirom da znamo da nikoje dve kraljice ne mogu biti u istoj koloni, možemo, bez gubitka opštosti, pretpostaviti da je prva kraljica u prvoj koloni, druga u drugoj, i sl. Time se gubi potreba za y promenljivama, a ostaju nam samo x promenljive i ograničenja:

- `alldiff`$(x_1, \ldots, x_8)$ – *nikoje dve kraljice se ne smeju nalaziti u istoj vrsti*
- *za* $i \neq j$ *mora da važi* $|x_i - x_j| \neq |i - j|$ *(primetimo da je* $|i - j|$ *konkretan broj za svako* i, j*)*

Transformacija ograničenja

Koliko slobode imamo u zadavanju ograničenja?

- CSP rešavači obično podržavaju ograničen skup tipova ograničenja
- Sva ograničenja koja se javljaju u modelu moraju se svesti na ona ograničenja koja rešavač podržava
- Ograničenje oblika $|x - y| \neq c$ obično nije među njima

Napomena

- Mi ćemo u nastavku ograničenja iz naših modela uvek svoditi na standardna ograničenja koja su podržana od strane većine rešavača
- Ovakva ograničenja ćemo, kao i do sada, eksplicitno uvoditi definicijama

Apsolutna vrednost

Definicija 6

Ograničenje oblika $y = |x|$ nazivamo ograničenjem apsolutne vrednosti.

Napomena

Ograničenje oblika $|x - y| \neq c$ se može svesti na dva ograničenja: $x - y \neq c$ i $x - y \neq -c$. Alternativno, možemo uvesti nove promenljive z i u i svesti polazno ograničenje na ograničenja: $z = x - y$, $u = |z|$ i $u \neq c$. Na sličan način se može transformisati i ograničenje oblika $|x - y| \neq |x' - y'|$.

Primer – 4 kraljice i 4 skakača

Primer 10

Razmotrimo sada sledeći problem: potrebno je rasporediti 4 kraljice i 4 skakača na šahovsku tablu tako da se međusobno ne napadaju.

Primer – 4 kraljice i 4 skakača

Primer (nastavak)

Slično kao i kod problema 8 kraljica, i ovde ćemo imati promenljive $x_1^Q, \ldots, x_4^Q$ *(vrste za kraljice),* $y_1^Q, \ldots, y_4^Q$ *(kolone za kraljice),* $x_1^N, \ldots, x_4^N$ *(vrste za skakače) i* $y_1^N, \ldots, y_4^N$ *(kolone za skakače). Sve ove promenljive imaće domen* $D = \{1, \ldots, 8\}$*. Imaćemo sledeća ograničenja:*

- *da se kraljice međusobno ne napadaju kodiramo slično kao i ranije:* $\texttt{alldiff}(x_1^Q, \ldots, x_4^Q)$, $\texttt{alldiff}(y_1^Q, \ldots, y_4^Q)$, *kao i* $|x_i^Q - x_j^Q| \neq |y_i^Q - y_j^Q|$ *za* $i \neq j$.
- *slično kodiramo da kraljice ne napadaju skakače:* $x_i^Q \neq x_j^N$ *i* $y_i^Q \neq y_j^N$ *za svako* i, j*, kao i* $|x_i^Q - x_j^N| \neq |y_i^Q - y_j^N|$*, za svako* i, j
- *da skakači ne napadaju kraljice:* $|x_i^Q - x_j^N| \cdot |y_i^Q - y_j^N| \neq 2$*, za svako* i, j
- *da se skakači međusobno ne napadaju:* $|x_i^N - x_j^N| \cdot |y_i^N - y_j^N| \neq 2$*, za* $i \neq j$
- *da se nikoja dva skakača ne nalaze na istom polju:* $x_i^N \neq x_j^N \vee y_i^N \neq y_j^N$*, za* $i \neq j$

Ograničenja proizvoda i stepena

Definicija 7

Ograničenje oblika

$$y = x_1 \cdot \ldots \cdot x_n$$

nazivamo ograničenje proizvoda. Slično, ograničenje:

$$y = x^n$$

nazivamo ograničenje stepena.

Napomena

U prethodnom primeru, ograničenje oblika $|x_1 - y_1| \cdot |x_2 - y_2| \neq c$ se može svesti na ograničenja $z_1 = x_1 - y_1$, $z_2 = x_2 - y_2$, $u_1 = |z_1|$, $u_2 = |z_2|$, $w = u_1 \cdot u_2$, $w \neq c$.

Optimizaciona varijanta CSP problema

Definicija 8

Problem ograničene optimizacije (engl. *Constrained Optimization Problem (COP)*) *je uređeni par* $(\mathcal{P}, f)$, *gde je* $\mathcal{P}$ *CSP problem nad promenljivama* $x_1, \ldots, x_n$ *sa domenima* $D_i = D(x_i)$, *a* $f : D_1 \times \ldots \times D_n \to \mathbb{R}$ *je funkcija cilja* (engl. *objective function*). *Rešenje COP problema je bilo koje rešenje CSP problema* $\mathcal{P}$ *koje maksimizuje (ili minimizuje) funkciju* f.

Primer – Problem ranca

Primer 11

Razmotrimo čuveni problem ranca (engl. knapsack problem): na raspolaganju nam je ranac fiksiranog kapaciteta K, kao i skup predmeta, pri čemu svaki predmet ima svoju veličinu i svoju vrednost. Zadatak je spakovati što više predmeta u ranac tako da se ne prekorači zadati kapacitet, a da vrednost spakovanih predmeta bude što veća.

Primer – Problem ranca (2)

Primer (nastavak)

Problem ranca se može predstaviti kao CSP problem na sledeći način: numerišimo najpre predmete redom sa $1, 2 \ldots, n$. *Neka je* v_i *vrednost* i*-tog predmeta, a* s_i *njegova veličina. Uvedimo promenljive* $x_1, \ldots, x_n$ *sa domenima* $D = \{0, 1\}$. *Intuitivno, promenljiva* x_i *biće jednaka* 1 *akko je* i*-ti predmet spakovan u ranac. Sada imamo ograničenje:*

$$s_1 x_1 + s_2 x_2 + \ldots + s_n x_n \leq K$$

kojim se obezbeđuje da se kapacitet ranca ne prekorači. Potrebno je maksimizovati funkciju:

$$f(x_1, \ldots, x_n) = v_1 x_1 + v_2 x_2 + \ldots + v_n x_n$$

Primer – Golombov lenjir

Primer 12

Razmotrimo problem Golombovog lenjira: za date prirodne brojeve n i k potrebno je odrediti niz od k celih brojeva $x_1 < x_2 < \ldots < x_k$ takav da su sve razlike $x_j - x_i$ (za $i < j$) međusobno različite, kao i da najveća među njima $x_k - x_1$ bude jednaka n.
NAPOMENA: U zavisnosti od n i k, problem može da ima nula ili više rešenja.

Primer – Golombov lenjir (2)

Primer (nastavak)

Problem Golombovog lenjira se može predstaviti na sledeći način:

- *imamo promenljive* $x_1, \ldots, x_k$ *sa domenom* $D = \{0, 1, \ldots, n\}$
- *imamo promenljive* y_{ij} $(1 \leq i < j \leq k)$ *sa domenom* $D' = \{1, \ldots, n\}$
- *imamo ograničenja* $x_1 < x_2, x_2 < x_3, \ldots x_{k-1} < x_k$, $x_k - x_1 = n$
- *imamo ograničenja:* $y_{ij} = x_j - x_i$ $(1 \leq i < j \leq k)$
- *imamo ograničenje* $\texttt{alldiff}(y_{21}, y_{32}, y_{31}, \ldots, y_{k,k-1}, y_{k1})$

NAPOMENA: iz gornjih uslova i zadatog domena, jasno je da mora biti $x_1 = 0$ *i* $x_k = n$*. Ipak, i ova dva ograničenja se mogu zadati, iako su redundantna, jer ponekad mogu olakšati posao rešavaču.*

Primer – Golombov lenjir, optimizaciona varijanta

Primer 13

Optimizaciona varijanta Golombovog lenjira: za fiksirano k odrediti najmanji mogući prirodan broj n takvo da problem Golombovog lenjira ima rešenje.

Model je isti kao ranije, s tim što bismo za veličinu domena uzeli neko N dovoljno veliko da budemo sigurni da postoji rešenje (npr. $N = k \cdot (k+1)/2$), a umesto ograničenja $x_k - x_1 = n$ imali bismo ograničenje $x_1 = 0$. Cilj bi bio minimizovati funkciju $f(x_1, \ldots, x_k, y_{12}, \ldots, y_{k-1,k}) = x_k$

Primer – svi intervali

Primer 14

Da li je moguće odsvirati svih 12 tonova jedne oktave (c, c#, d, d#, itd.) u takvom redosledu da nikoja dva intervala između susednih odsviranih tonova ne budu jednaka?

NAPOMENA: ako tonove redom označimo brojevima $0, 1, \ldots, 11$*, tada je interval izmeđdva tona* x *i* y *jednak njihovoj apsolutnoj razlici* $|x - y|$ *(1 je poluton, a 11 je velika sedmica). Problem se sada može formulisati i ovako:*

Da li postoji permutacija $x_0 x_1 \ldots x_{11}$ *brojeva* $0, 1, \ldots, 11$*, takva da za* $y_i = |x_i - x_{i-1}|$ *niz brojeva* $y_1 y_2 \ldots y_{11}$ *predstavlja permutaciju brojeva* $1, 2, \ldots, 11$*?*

Primer – svi intervali (2)

Primer (nastavak)

Označimo promenljive kao na prethodnom slajdu. Domen x promenljivih će biti skup $\{0, 1, \ldots, 11\}$*, a domen y promenljivih će biti* $\{1, 2, \ldots, 11\}$*. Sada imamo ograničenja:*

- $y_i = |x_i - x_{i-1}|$, *za* $i \in \{1, \ldots, 11\}$
- $\texttt{alldiff}(x_0, \ldots, x_{11})$
- $\texttt{alldiff}(y_1, \ldots, y_{11})$

Primer – Langfordovi parovi

Primer 15

Za dato n, formirati permutaciju brojeva $1, 1, 2, 2, \ldots, n, n$ *takvu da između dve jedinice postoji jedan broj, između dve dvojke dva broja, i sl. Na primer, za* $n = 4$*, jedna moguća permutacija bila bi* $4, 1, 3, 1, 2, 4, 3, 2$.

Problem nema rešenje za svako n, dok za neke vrednosti n može imati i više različitih rešenja.

Primer – Langfordovi parovi (2)

Primer (nastavak)

Uvedimo promenljive x_1^1, x_1^2 *(pozicije prve i druge kopije jedinice u nizu),* x_2^1, x_2^2 *(pozicije prve i druge kopije dvojke u nizu), i sl. Sve ove promenljive (njih* $2n$*) imaju domen* $D = \{1, 2, \ldots, 2n\}$*.*
Pritom, važe ograničenja:

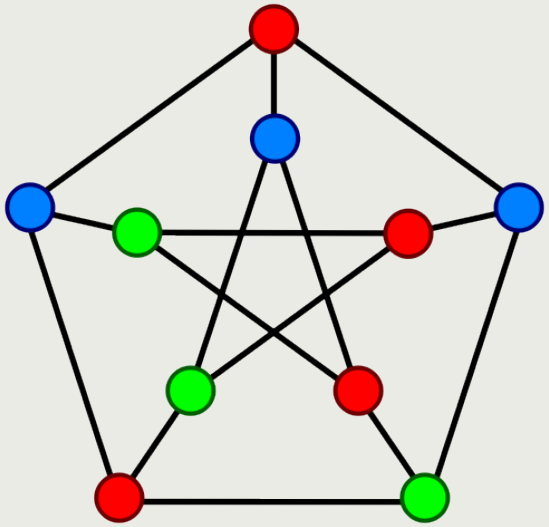
- $\texttt{alldiff}(x_1^1, x_1^2, x_2^1, x_2^2, \ldots, x_n^1, x_n^2)$ *(svi moraju biti na različitim pozicijama)*
- $x_1^2 - x_1^1 = 1$, $x_2^2 - x_2^1 = 2$, $\ldots$, $x_n^2 - x_n^1 = n$

Primetimo da ne koristimo apsolutne vrednosti, npr. $|x_1^2 - x_1^1| = 1$*, jer želimo da razbijemo simetrije (zahtevamo da druga kopija bude desno od prve).*

Primer – bojenje grafa

Primer 16

Obojiti čvorove grafa pomoću n različitih boja tako da nikoja dva susedna čvora ne budu obojena istom bojom.
Na sledećoj slici dat je primer ispravno obojenog grafa sa 3 boje.



NAPOMENA: Za $n \geq 3$ problem bojenja grafa je NP-kompletan.

Primer – bojenje grafa (2)

Primer (nastavak)

Problem bojenja grafa se može jednostavno predstaviti kao CSP problem, tako što se uvede po jedna promenljiva za svaki čvor, sa domenom $\{1, 2, \ldots, n\}$ *(intuitivno, vrednost te promenljive predstavlja boju čvora). Sada treba uvesti ograničenja različitosti* $x_i \neq x_j$ *za svaka dva susedna čvora* i *i* j*.*

Alternativno, ako znamo da neki skup čvorova $\{x_{i_1}, x_{i_2}, \ldots, x_{i_k}\}$ *čine kliku*[1] *veličine* k*, tada uslove različitosti među njima možemo kompaktno predstaviti ograničenjem* $\texttt{alldiff}(x_{i_1}, x_{i_2}, \ldots, x_{i_k})$*.*

[1] Klika je skup čvorova grafa takav da između svaka dva čvora postoji grana

Primer – maksimalna klika

Primer 17

Razmotrimo problem maksimalne klike: u datom grafu pronaći kliku maksimalne veličine.

Primer – maksimalna klika (2)

Primer (nastavak)

Za svaki od čvorova grafa $v_1, \ldots, v_n$ *uvedimo po jednu promenljivu* $x_1, \ldots, x_n$ *sa domenom* $\{0, 1\}$. *Intuitivno,* $x_i = 1$ *akko je* v_i *član maksimalne klike. Sada imamo ograničenja oblika* $x_i \neq 1 \vee x_j \neq 1$ *za svaka dva čvora* v_i *i* v_j *za koje ne postoji grana između njih. Cilj je maksimizovati funkciju:*

$$f(x_1, \ldots, x_n) = x_1 + x_2 + \ldots + x_n$$

Disjunktivno ograničenje i ograničenje negacije

Definicija 9

Ograničenje oblika $C_1 \vee \ldots \vee C_k$*, gde su* $C_1, \ldots, C_k$ *takođe ograničenja nazivamo disjunktivno ograničenje.*
Ograničenje oblika $\neg C$ *gde je* C *ograničenje nazivamo ograničenje negacije.*

Napomena

- Disjunktivno ograničenje i ograničenje negacije su zapravo konstruktori kojima se kreiraju nova ograničenja od postojećih ograničenja
- Drugi logički veznici poput implikacije i ekvivalencije se mogu svesti na ova ograničenja:
  - $C_1 \Rightarrow C_2$ se svodi na $\neg C_1 \vee C_2$
  - $C_1 \Leftrightarrow C_2$ se svodi na dva ograničenja: $\neg C_1 \vee C_2$ i $C_1 \vee \neg C_2$
  - *if* C *then* C_1 *else* C_2 se svodi na ograničenja: $\neg C \vee C_1$ i $C \vee C_2$

Primer – sudoku

Primer 18

Problem sudoku slagalice je uopštenje problema latinskog kvadrata, sa dodatnim uslovima različitosti po „kvadratima". Primer jedne sudoku slagalice dat je na sledećoj slici.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

U formi CSP problema, imaćemo po jednu promenljivu x_{ij} za svako polje (i, j) sa domenom $\{1, 2, \ldots, 9\}$, kao i ograničenja `alldiff` *po vrstama, kolonama i 3×3 kvadratima. Takođe, imaćemo jednakosti koje predstavljaju zadate vrednosti (npr. $x_{11} = 5$).*

Primer – društveni golferi

Primer 19

Grupa od $m \cdot n$ igrača jednom nedeljno igra golf. Dele se u m ekipa po n igrača. Kako bi se što više družili, cilj im je da svaki put menjaju ekipe, tako da nikoja dva igrača ne igraju dva puta u istom timu. Potrebno je napraviti takav raspored za p nedelja. Da li je to moguće?

Odgovor na ovo pitanje zavisi od parametara m, n, p. U pitanju je težak kombinatorni problem iz klase problema raspoređivanja.

Primer – društveni golferi (2)

Primer (nastavak)

Problem društvenih golfera se može modelovati kao CSP problem tako što najpre pretpostavimo da su igrači unapred numerisani brojevima $0, 1, \ldots, mn - 1$, *a zatim uvedemo promenljivu* x_{ijk} *(*$i \in \{1, \ldots, m\}$, $j \in \{1, \ldots, n\}$, $k \in \{1, \ldots, p\}$*) sa domenom* $\{0, 1, \ldots, mn - 1\}$. *Intuitivno, vrednost promenljive* x_{ijk} *označava igrača koji* k*-te nedelje igra u* i*-toj ekipi na* j*-toj poziciji. Sada ćemo za svako fiksirano* k *imati ograničenje* `alldiff` *po svim* x_{ijk} *za* $i \in \{1, \ldots, m\}$, $j \in \{1, \ldots, n\}$ *(jer jedan isti igrač ne može biti na dve različite pozicije u isto vreme). U cilju razbijanja simetrija, pretpostavićemo da je za svako fiksirano* i *i* k *(tj. fiksiranu ekipu u fiksiranoj nedelji) imamo* $x_{i1k} < x_{i2k} < \ldots < x_{ink}$ *(tj. ne želimo da razmatramo različite permutacije iste ekipe).*

	Ekipa 1	*Ekipa 2*	...	*Ekipa m*
Nedelja 1	*1 2 3 4*	*5 6 7 8*	...	
Nedelja 2	*1 5 9 13*	*2 6 10 14*	...	
⋮	⋮	⋮	⋱	
Nedelja p				

Primer – društveni golveri (3)

Primer (nastavak)

Dalje, potrebno je obezbediti da nikoja dva igrača ne igraju više od jednom u istoj ekipi. Da bismo to postigli, primetimo bijektivno mapiranje:

$$(u, v) \mapsto mn \cdot u + v$$

skupa $\{0, 1, \ldots, mn - 1\} \times \{0, 1, \ldots, mn - 1\}$ *na skup* $\{0, 1, \ldots, m^2n^2 - 1\}$. *Intuitivno, ovo mapiranje svakom uređenom paru igrača pridružuje jedinstven kôd. Sada uvodimo promenljive* $y_{ij_1j_2k}$ *(za* $i \in \{1, \ldots, m\}$, $k \in \{1, \ldots, p\}$ *i* $1 \leq i_1 < i_2 \leq n$*) sa domenom* $\{0, 1, \ldots, m^2n^2 - 1\}$ *i zadajemo ograničenja* $y_{ij_1j_2k} = mn \cdot x_{ij_1k} + x_{ij_2k}$ *(tj. promenljiva* $y_{ij_1j_2k}$ *predstavlja kôd pridružen paru igrača koji igraju u* i*-toj ekipi na pozicijama* j_1 *i* j_2 *tokom* k*-te nedelje). Sada imamo jedno* `alldiff` *ograničenje nad svim* y *promenljivama (jer se ni jedan kôd ne sme pojaviti više od jednom, što bi značilo da se isti par igrača dva puta pojavio zajedno u ekipi).*

Primer – zebra

Primer 20

Imamo 5 kuća, jednu do druge: crvena, zelena, žuta, bela i plava, u nekom redosledu. Takođe, imamo 5 osoba: Englez, Španac, Ukrajinac, Norvežanin i Japanac koji žive u tim kućama (svako u svojoj). Ove osobe piju 5 različitih napitaka: kafu, čaj, mleko, orandžadu i limunadu. Takođe, svaka osoba poseduje jednog od sledećih kućnih ljubimaca: konja, psa, lisicu, zebru i puža (opet, svako ima različitog ljubimca). Najzad, svaka od osoba se bavi jednim od sledećih zanimanja (svaka različitim): programer, pevač, pekar, advokat i profesor. Ono što znamo je sledeće:

- *Englez živi u crvenoj kući.*
- *Španac poseduje psa.*
- *Kafu pije vlasnik zelene kuće.*
- *Ukrajinac pije čaj.*
- *Zelena kuća je odmah iza bele kuće.*
- *Programer poseduje puža.*
- *Pevač živi u žutoj kući.*
- *Mleko se pije u srednjoj kući.*
- *Norvežanin živi u prvoj kući.*
- *Pekar živi u kući pored kuće čoveka koji ima lisicu.*
- *Pevač živi u kući pored kuće čoveka koji ima konja.*
- *Advokat pije orandžadu.*
- *Japanac je profesor.*
- *Norvežanin živi pored plave kuće.*

Pitanje je: ko poseduje zebru?

Primer – zebra (2)

Primer (nastavak)

Jedno moguće rešenje se može predstaviti tabelom:

Kuća	**1**	**2**	**3**	**4**	**5**
Boja	*žuta*	*plava*	*crvena*	*bela*	*zelena*
Nacionalnost	*Norvežanin*	*Ukrajinac*	*Englez*	*Španac*	*Japanac*
Napitak	*limunada*	*čaj*	*mleko*	*orandžada*	*kafa*
Zanimanje	*pevač*	*pekar*	*programer*	*advokat*	*profesor*
Ljubimac	*lisica*	*konj*	*puž*	*pas*	**zebra**

Primer – zebra (3)

Primer (nastavak)

Da bismo ovaj problem predstavili kao CSP, uvedimo promenljive zuta, plava, . . . , Englez, Norvezanin,. . . , limunada, caj, . . . , pevac, pekar, . . . , lisica, konj, . . . , pri čemu svaka od ovih promenljivih ima domen $\{1, 2, \ldots, 5\}$. *Intuitivno, vrednost svake promenljive predstavlja redni broj kuće za koju se vezuje odgovarajuća osoba, profesija, ljubimac, i sl. Sada se ograničenja mogu lako izraziti:*

- $Englez = crvena$
- $Spanac = pas$
- $kafa = zelena$
- $Ukrajinac = caj$
- $zelena = bela + 1$
- $programer = puz$
- $pevac = zuta$
- $mleko = 3$
- $Norvezanin = 1$
- $|pekar - lisica| = 1$
- $|pevac - konj| = 1$
- $advokat = orandzada$
- $Japanac = profesor$
- $|Norvezanin - plava| = 1$

Dodatno, imamo ograničenja $\texttt{alldiff}(Englez, \ldots, Japanac)$, $\texttt{alldiff}(zuta, \ldots, bela)$, $\texttt{alldiff}(pevac, \ldots, profesor)$, $\texttt{alldiff}(kafa, \ldots, limunada)$ *i* $\texttt{alldiff}(pas, \ldots, zebra)$, *kako bismo obezbedili različitost.*

Primer – Neptunove sluge

Primer 21

Među Neptunovim slugama nalaze se oktopodi sa po 6, 7 ili 8 nogu. Oni koji imaju 7 nogu uvek lažu, dok oni koji imaju 6 ili 8 nogu uvek govore istinu. Četiri oktopoda, Plavi, Crveni, Zeleni i Žuti su se jednom prilikom srela. Tada je Plavi izjavio: ,,Nas četvorica zajedno imamo 28 nogu". Zeleni je izjavio: ,,Nas četvorica zajedno imamo 27 nogu". Žuti je izjavio: ,,Nas četvorica zajedno imamo 26 nogu". Crveni je izjavio: ,,Nas četvorica zajedno imamo 25 nogu". Koliko nogu ima koji oktopod?

Primer – Neptunove sluge (2)

Primer (nastavak)

Označimo sa B, R, G, Y redom promenljive koje označavaju broj nogu Plavog, Crvenog, Zelenog i Žutog oktopoda. Svaka od ovih promenljivih ima domen $\{6, 7, 8\}$. Neka je S ukupan broj nogu svih oktopoda (ova promenljiva može imati domen $\{24, \ldots, 32\}$). Pored ograničenja $S = B + R + G + Y$, imaćemo i ograničenja:

- $B = 7 \Leftrightarrow S \neq 28$
- $R = 7 \Leftrightarrow S \neq 25$
- $G = 7 \Leftrightarrow S \neq 27$
- $Y = 7 \Leftrightarrow S \neq 26$

Primer – raspored časova

Primer 22

Posmatrajmo školu koja ima 40 odeljenja, pri čemu svako odeljenje pohađa 35 časova nedeljno, raspoređenih u pet radnih dana (7 časova dnevno). Svakoga dana postoji tačno sedam 45-minutnih termina na koje je potrebno rasporediti časove. Neki časovi se moraju držati u blokovima od po 2 ili 3 susedna časa istog dana. Nastavu drži određeni broj nastavnika, pri čemu svaki od njih ima od 14 do 20 časova nedeljno. Svaki od časova svakog odeljenja je unapred dodeljen konkretnom nastavniku. Zadatak je formirati raspored časova.

Primer – raspored časova (2)

Primer (nastavak)

Numerišimo najpre sve termine redom brojevima $1, 2, \ldots, 35$*:*

Ponedeljak	**Utorak**	**Sreda**	**Četvrtak**	**Petak**
1	*8*	*15*	*22*	*29*
2	*9*	*16*	*23*	*30*
3	*10*	*17*	*24*	*31*
4	*11*	*18*	*25*	*32*
5	*12*	*19*	*26*	*33*
6	*13*	*20*	*27*	*34*
7	*14*	*21*	*28*	*35*

Primer – raspored časova (3)

Primer (nastavak)

Numerišimo, dalje, nastavnike, grupe i njihove časove. Na primer:

Nastavnik	**Kôd**
Petar Petrović	*1*
Dejan Mikić	*2*
⋮	⋮
Jovan Jovanović	*12*

Odeljenje	**Kôd**
I_1	*1*
I_2	*2*
⋮	⋮
IV_{10}	*40*

Čas odeljenja I_1	**Kôd**
Matematika	*1*
Matematika	*2*
Matematika	*3*
Matematika	*4*
Srpski	*5*
Srpski	*6*
Srpski	*7*
Fizičko	*8*
Fizičko	*9*
⋮	⋮
Filozofija	*35*

Primer – raspored časova (4)

Primer (nastavak)

Pretpostavimo da je dodela časova nastavnicima, zajedno sa uslovima za blokove data u sledećoj tabeli:

Kôd odeljenja	**Kôd časa**	**Kôd nastavnika**	**Blok**
1	1	1	1
1	2	1	1
1	3	1	2
1	4	1	2
1	5	2	3
1	6	2	3
1	7	2	3
1	8	7	4
1	9	7	5
⋮	⋮	⋮	⋮
1	35	12	16

Primer – raspored časova (5)

Primer (nastavak)

Da bismo opisani problem predstavili kao CSP problem, uvedimo promenljive x_{ijk} za i-tog nastavnika koji drži k-ti čas j-tom odeljenju. Domen ovih promenljivih biće skup $\{1, \ldots, 35\}$, a vrednost promenljive x_{ijk} intuitivno predstavlja redni broj termina u kome se taj čas održava.

- *da bismo obezbedili da i-ti nastavnik ne drži dva časa u isto vreme, zadajemo ograničenje* `alldiff` *nad svim x_{ijk} promenljivama za to fiksirano i*
- *da bismo obezbedili da j-ta grupa nema dva časa u isto vreme, zadajemo ograničenje* `alldiff` *nad svim x_{ijk} za to fiksirano j*
- *da bismo izrazili uslove vezane za blokove, za svaka dva časa k_1 i k_2 koji bi trebalo da budu susedni u bloku zadajemo ograničenje $x_{ijk_1} + 1 = x_{ijk_2}$*
- *kako se blokovi ne bi „razbili" u dva dana, ako npr. k-ti čas nije poslednji čas u svom bloku, dodajemo uslov da on ne može biti poslednji čas nekog dana: $x_{ijk} \bmod 7 \neq 0$.*

Ograničenja *div* i *mod*

Definicija 10

Neka su x, y, z promenljive. Ograničenja:

$$x = y \ div \ z$$

odnosno

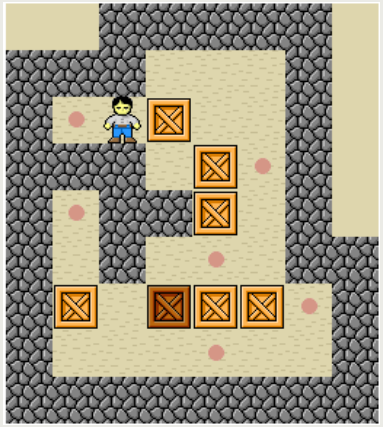
$$x = y \ mod \ z$$

nazivamo ograničenjima celobrojnog količnika i ostatka pri deljenju, respektivno. Umesto promenljive x može da stoji i konstanta c.

Primer – Sokoban

Primer 23

Sokoban je „čuvar skladišta" koji ima zadatak da napakuje sanduke na obeležena polja. Pritom, on ne može prolaziti kroz zidove i može gurati samo po jedan sanduk (ne može vući sanduke niti gurati više od jednog sanduka odjednom).



Pitanje: za dato N, da li je moguće rešiti dati Sokoban problem u N koraka?

Primer – Sokoban (2)

Primer (nastavak)

Pretpostavimo da se tabla za igru predstavlja matricom $m \times n$. Na svakom polju table se nalazi ili zid, ili odredište, ili ni jedno od ta dva. Ova konfiguracija je fiksirana i ne menja se tokom igre. Imamo sledeće promenljive (tj. matrice promenljivih):

- $wall[i, j]$ *sa domenom* $\{true, false\}$ *koja označava da li se na poziciji* (i, j) *nalazi zid*
- $home[i, j]$ *sa domenom* $\{true, false\}$ *koja označava da li je na poziciji* (i, j) *odredište*

Ove promenljive imaju unapred zadate vrednosti (ograničenjima oblika npr. $wall[i, j] = true$ *ili* $home[i, j] = false$*).*
Ako se na polju ne nalazi zid, tada se na tom polju može nalaziti ili kutija, ili Sokoban, ili ni jedno od ta dva. Imamo sledeće promenljive:

- $box_k[i, j]$ *sa domenom* $\{true, false\}$ *koja označava da se nakon* k *koraka na poziciji* (i, j) *nalazi kutija* $(k \in \{0, \ldots, N\})$
- I_k *sa domenom* $\{1, 2, \ldots, m\}$ *i* J_k *sa domenom* $\{1, 2, \ldots, n\}$ *koje označavaju, respektivno, redni broj vrste i kolone u kojoj se nalazi Sokoban nakon* k*-tog koraka* $(k \in \{0, \ldots, N\})$

Najzad, imamo promenljivu $move_k$ *sa domenom* $\{up, down, left, right\}$ *koja označava potez u* k*-tom koraku* $(k \in \{1, \ldots, N\})$*.*

Primer – Sokoban (3)

Primer (nastavak)

Početno stanje igre kodiramo uslovima nad promenljivama $box_0[i, j]$, K_0 *i* J_0. *Na primer, u našem primeru sa slike imamo da se Sokoban na početku nalazi na poziciji* $(3, 3)$, *pa ćemo imati ograničenja* $I_0 = 3$ *i* $J_0 = 3$. *Slično, na polju* $(3, 4)$ *imamo kutiju, pa ćemo imati uslov* $box_0[3, 4] = true$, *i td.*

Primer – Sokoban (4)

Primer (nastavak)

Da bismo garantovali da se odgovarajući potez može primeniti u k-tom koraku, moramo zadati ograničenja za tekuće $(k-1)$-vo stanje. Na primer, za potez „na gore":

$$move_k = up \Rightarrow \begin{pmatrix} I_{k-1} > 1 \wedge \\ wall[I_{k-1}-1, J_{k-1}] = false \wedge \\ box_{k-1}[I_{k-1}-1, J_{k-1}] = true \Rightarrow \\ I_{k-1} > 2 \wedge \\ box_{k-1}[Y_{k-1}-2, J_{k-1}] = false \wedge \\ wall[I_{k-1}-2, J_{k-1}] = false \end{pmatrix}$$

Intuitivno, da bismo mogli da idemo na gore, ne smemo biti u prvoj vrsti, u vrsti iznad ne sme biti zid, a ako je iznad nas kutija, tada i iznad nje ne sme biti ni zid ni kutija.

Primer – Sokoban (5)

Primer (nastavak)

Da bismo opisali efekat k-tog koraka, moramo uspostaviti odnos između tekućeg $(k-1)$-vog i narednog k-tog stanja. Ovaj efekat zavisi od poteza koji je izvršen u k-tom koraku. Na primer, za potez ,,na gore", imamo sledeće ograničenje:

$$move_k = up \Rightarrow \begin{array}{c} I_k = I_{k-1} - 1 \wedge \\ J_k = J_{k-1} \wedge \\ \left(\begin{array}{c} box_{k-1}[I_{k-1}-1, J_{k-1}] = false \Rightarrow \\ \forall\ i,j.\ box_k[i,j] = box_{k-1}[i,j] \end{array} \right) \wedge \\ \left(\begin{array}{c} box_{k-1}[I_{k-1}-1, J_{k-1}] = true \Rightarrow \\ \forall\ i,j.\ box_k[i,j] = box'_{k-1}[i,j] \end{array} \right) \end{array}$$

pri čemu je sa $box'_{k-1}[i,j]$ označena matrica koja se poklapa sa $box_{k-1}[i,j]$ na svim pozicijama osim što su vrednosti na pozicijama $(I_{k-1}-1, J_{j-1})$ i $(I_{k-1}-2, J_{k-1})$ razmenjene (ovaj uslov treba raspisati, ali to ovde ne činimo zbog nedostatka prostora). Intuitivno, ako je k-ti potez ,,na gore", tada se Sokoban u sledećem stanju pomera za jednu vrstu na gore (ostaje u istoj koloni), pri čemu ako iznad nije bilo kutije, sve kutije ostaju na svojim mestima, a u suprotnom se ta jedna kutija pomera za jednu poziciju na gore.

Primer – Sokoban (6)

Primer (nastavak)

Najzad, potrebno je da nakon N koraka sve kutije budu na svom mestu:

$$\forall\ i, j.\ box_N[i, j] = home[i, j]$$

Napomena

Zapis $\forall\ i, j. C(i, j)$ označava da se radi o skupu ograničenja $C(i, j)$ dobijenih iteracijom i i j kroz skupove $\{1, \ldots, m\}$ i $\{1, \ldots, n\}$, respektivno. U većini jezika za modelovanje, ovo „raspisivanje" se obavlja automatski.

Nizovi i matrice

Promenljive nizovskog tipa

- Promenljive koje predstavljaju nizove se veoma često javljaju u modelima
- Ipak, nizovi obično nisu direktno podržani od strane CSP rešavača
- Promenljiva nizovskog tipa $x[1..n]$ se obično prevodi u niz pojedinačnih promenljivih $x_1, \ldots, x_n$
  - višedimenzioni nizovi (matrice) se svode na jednodimenzione: $x[i,j] \rightsquigarrow x[n \cdot i + j]$
- Svako pojavljivanje $x[i]$ (gde je i konstanta) u modelu se prevodi u x_i
- Problem: šta ako i nije konstanta već promenljiva?

Definicija 11

Ograničenje $\texttt{element}(y, x_1, \ldots, x_n, v)$, *gde su* $y, x_1, \ldots, x_n, v$ *promenljive, se definiše na sledeći način:*

$$\texttt{element}(y, x_1, \ldots, x_n, v) = \{(i, d_1, \ldots, d_n, d_i) \mid 1 \leq i \leq n\}$$

Napomena

Intuitivno, $\texttt{element}(y, x_1, \ldots, x_n, v)$ označava da je $v = x_y$.

Primer – raspored časova (ponovo)

Primer 24

Razmotrimo ponovo primer sa rasporedom časova. Pretpostavimo da imamo dodatni uslov da ni jedan nastavnik ne sme da drži više od 4 vezana časa. Ovo bismo mogli kodirati tako što za i-tog nastavnika uvedemo nizovsku promenljivu $R_i[1..35]$*, čiji elementi imaju domen* $\{true, false\}$*, pri čemu je* $R_i[k] = true$ *akko i-ti nastavnik drži čas u k-tom terminu. Ovaj uslov možemo nametnuti ograničenjima* $R_i[x_{ijk}] = true$ *za sve promenljive* x_{ijk} *za to fiksirano i (izraženo pomoću element ograničenja). Sada za npr. ponedeljak možemo zadati ograničenje:*

$$\texttt{stretch}(R_i[1], \ldots, R_i[7], \{(true, 1, 4)\})$$

Ovo ograničenje zahteva da u nizu promenljivih $R_i[1], \ldots, R_i[7]$ *dužina svake sekvence uzastopnih promenljivih sa vrednošću true bude između* 1 *i* 4*. Slična ograničenja treba zadati i za ostale dane.*

Ograničenje stretch

Definicija 12

Ograničenje `stretch` *definišemo na sledeći način:*

$$\texttt{stretch}(x_1, \ldots, x_n, \left\{ \begin{array}{c} (c_1, l_1, u_1), \\ \ldots, \\ (c_k, l_k, u_k) \end{array} \right\}) =$$

$$\left\{ (d_1, \ldots, d_n) \mid d_i \in D(x_i), \forall i, j.\ d_i = c_j \Rightarrow \exists p, q. \begin{array}{c} p \leq i \leq q \wedge \\ l_j \leq q - p + 1 \leq u_j \wedge \\ \forall k \in \{p, \ldots, q\}. d_k = c_j \end{array} \right\}$$

gde su $x_1, \ldots, x_n$ *promenljive, a* c_j, l_j, u_j *konstante.*

Napomena

Intuitivno, svaku od vrednosti c_j mogu uzeti najmanje l_j a najviše u_j uzastopnih promenljivih u nizu $x_1, \ldots, x_n$.

Primer – raspored časova (ponovo)

Primer 25

Pretpostavimo dodatni uslov: nastavnici ne vole da ustaju rano, pa želimo da obezbedimo da svaki nastavnik najviše dva puta nedeljno ima prvi čas. Uvedimo najpre promenljivu c_{ijk} *koja označava redni broj časa u toku dana kada* i*-ti nastavnik drži* k*-ti čas* j*-tom odeljenju. Ovo obezbeđujemo nametanjem ograničenja* $c_{ijk} = x_{ijk} \bmod 7$*. Sada možemo zadati ograničenja:*

$$\texttt{atmost}(c_{ij_1k_1}, c_{ij_2k_2}, \ldots, 2, 1)$$

čime se zahteva da, za svako fiksirano i*, od svih promenljivih* c_{ijk} *najviše dve imaju vrednost* 1*.*

Ograničenja kardinalnosti

Definicija 13

Ograničenje $\texttt{atleast}(x_1, \ldots, x_n, l, c)$*, gde su* $x_1, \ldots, x_n$ *promenljive, a* l *i* c *konstante, definišemo na sledeći način:*

$$\texttt{atleast}(x_1, \ldots, x_n, l, c) = \{(d_1, \ldots, d_n) \mid |\{i \mid d_i = c\}| \geq l\}$$

Slično, ograničenje $\texttt{atmost}(x_1, \ldots, x_n, u, c)$ *definišemo na sledeći način:*

$$\texttt{atmost}(x_1, \ldots, x_n, u, c) = \{(d_1, \ldots, d_n) \mid |\{i \mid d_i = c\}| \leq u\}$$

Napomena

Ograničenja kardinalnosti nameću gornju, odnosno donju, granicu za broj promenljivih koje uzimaju neku zadatu vrednost.

Primer – raspored časova (ponovo)

Primer 26

Pretpostavimo da, dodatno, želimo da osiguramo da svaki nastavnik svaki dan ima bar po jedan čas. Slično kao i malopre, uvedimo promenljivu d_{ijk} koja označava dan (1 za ponedeljak, 5 za petak) kada i-ti nastavnik drži k-ti čas j-tom odeljenju. Ovo obezbeđujemo ograničenjima $d_{ijk} = (x_{ijk} - 1)$ div $7 + 1$. Sada možemo nametnuti ograniěnja:

$$\begin{array}{l} atleast(d_{ij_1k_1}, d_{ij_2k_2}, \ldots, 1, 1) \\ atleast(d_{ij_1k_1}, d_{ij_2k_2}, \ldots, 1, 2) \\ \vdots \\ atleast(d_{ij_1k_1}, d_{ij_2k_2}, \ldots, 1, 5) \end{array}$$

Drugim rečima, bar jedan čas je raspoređen u ponedeljak, bar jedan u utorak, i td.

Ograničenje globalne kardinalnosti

Primer (nastavak)

Alternativno, umesto pojedinačnih `atleast` *ograničenja, možemo nametnuti jedno ograničenje:*

$$\mathrm{gcc}(d_{ij_1k_1}, d_{ij_2k_2}, \ldots, \{(1,1,7),(2,1,7),\ldots,(5,1,7)\})$$

Definicija 14

Ograničenje globalne kardinalnosti gcc *definišemo na sledeći način:*

$$\mathrm{gcc}(x_1,\ldots,x_n,\left\{\begin{array}{l}(c_1,l_1,u_1),\\ \ldots,\\ (c_k,l_k,u_k)\end{array}\right\}) = \{(d_1,\ldots,d_n) \mid \forall j.l_j \leq |\{d_i \mid d_i = c_j\}| \leq u_j\}$$

Dakle, za svaku vrednost c_j zahtevamo da broj promenljivih koje uzimaju tu vrednost pripada intervalu $[l_j, u_j]$.

Primer – raspoređivanje procesa

Primer 27

Razmotrimo problem raspoređivanje poslova: imamo m izvršioca i n poslova koje treba završiti. Svaki posao ima svoje trajanje. Postoje međuzavisnosti između poslova – neki poslovi ne mogu početi pre nego što se neki drugi završe. Takođe, imamo r različitih (obnovljivih) resursa, pri čem svaki resurs ima ograničenu količinu. Za svaki posao se zna kolika mu je količina kog resursa potrebna. Potrebno je rasporediti poslove na m izvršioca tako da ukupno vreme za završetak svih poslova bude minimalno. Pritom, ni u jednom trenutku ne sme biti prekoračena maksimalna količina svakog resursa.

Primer – raspoređivanje procesa

Primer (nastavak)

Pretpostavimo da je trajanje izraženo u celim vremenskim jedinicama (npr. u satima). Neka je l_i konstanta koja predstavlja trajanje i-tog posla. Neka promenljiva x_i označava vremensku jedinicu kada počinje i-ti posao da se izvršava. Ove promenljive mogu imati domen $\{1, \ldots, N\}$, gde je N jednako zbiru dužina svih poslova (time je garantovano da će biti rešenja, makar se izvršavali sekvencijalno). Neka promenljiva y_{ij} (sa domenom $\{0, 1\}$) ima vrednost 1 akko i-ti posao traje tokom j-tog sata. Ovo postižemo ograničenjem: $y_{ij} = 1 \Leftrightarrow x_i \leq j \leq x_i + l_i - 1$. Sada ćemo, za svako $j \in \{1, \ldots N\}$, imati ograničenje:

$$y_{1j} + y_{2j} + \ldots + y_{nj} \leq m$$

kako bismo obezbedili da najviše m poslova budu aktivni u svakom trenutku. Slično, ako za svaki resurs k njegov limit označimo sa L_k, a količinu k-tog resursa koja je potrebna za i-ti posao označimo sa R_{ik}, imaćemo ograničenja (za svako j, k):

$$R_{1k} \cdot y_{1j} + \ldots + R_{nk} \cdot y_{nj} \leq L_k$$

Najzad, za svaka dva posla i_1 i i_2 takvih da i_2 zavisi od i_1, imaćemo ograničenje: $x_{i_2} \geq x_{i_1} + l_{i_1}$. Da bismo optimizovali ukupno vreme izvršavanja poslova, uvešćemo i promenljivu D, uz ograničenja: $x_i + l_i - 1 \leq D$, za svako i. Sada je zadatak minimizovati funkciju:

$$f(x_1, \ldots, x_n, y_{11}, \ldots, y_{nN}, D) = D$$